Kazakh-British Technical University

Data collection and preparation

Laboratory work №3

Data Transformation & EDA: Pandas GroupBy, joins, reshaping (pivot/melt) + Matplotlib/Seaborn plots.

Full Name:

Baibolov Sultanmakhmut

ID:

23B031235

29.09.2026

Almaty

Table of contents

Task 1. GroupBy 3
Task 2. Joins 4
Task 3. Reshaping 5
Task 4. Visualization 6
Task 5. EDA 7

Task 1. GroupBy

Use a dataset containing numerical and categorical variables. Group the data by one categorical column and calculate several aggregate statistics. Then group the data by two categorical columns and create a summary table using multiple aggregation functions.

I imported the necessary libraries and created a DataFrame from the CSV file. Then I checked the dataset for missing data and removed the rows with it. After that, I changed the data types for the columns from string to the corresponding categories and numeric types using the helper functions parse_* and apply.

Next, I did grouping with 1 key of fuel type displaying some aggregation data in selling price and the same with 2 keys – fuel type and transmission.

Data clearing

[111]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   name           8128 non-null   str
 1   year           8128 non-null   int64
 2   selling_price  8128 non-null   int64
 3   km_driven      8128 non-null   int64
 4   fuel           8128 non-null   str
 5   seller_type    8128 non-null   str
 6   transmission   8128 non-null   str
 7   owner          8128 non-null   str
 8   mileage        7907 non-null   str
 9   engine         7907 non-null   str
 10  max_power      7913 non-null   str
 11  torque         7906 non-null   str
 12  seats          7907 non-null   float64
dtypes: float64(1), int64(3), str(9)
memory usage: 825.6 KB
```

[112]:
```
df = df.drop(columns=['torque', 'seats'])
```

[113]:
```
df = df.dropna()
```

Changing types and normalization

[114]:
```
km_kg_to_kmpl = {
    'LPG': 0.54,
    'CNG': 0.72,
}
def parse_mileage(row: pd.Series) -> str:
    # if (pd.isna(row['mileage'])):
    #     return row
    if row['mileage'].endswith(' km/kg'):
        value = float(row['mileage'].rstrip(' km/kg'))
        value *= km_kg_to_kmpl[row['fuel']]
    else:
        value = row['mileage'].rstrip(' kmpl')
    row['mileage'] = value
    return row
```

[115]:
```
df = df.apply(parse_mileage, axis=1)
df['mileage'] = df['mileage'].astype('float64')
```

[116]:
```
def parse_engine(engine: str):
    return engine.rstrip(" C")
```

[117]:
```
df['engine'] = df['engine'].apply(parse_engine).astype('int64')
```

[118]:
```
categories = ['fuel', 'seller_type', 'transmission', 'owner']
for category in categories:
    df[category] = df[category].astype('category')
```

Grouping by fuel

[121]:
```
df.groupby(by="fuel")[["km_driven", "selling_price"]].agg([
    'mean', 'max', 'min'
])
```

[121]:

	km_driven			selling_price		
	mean	max	min	mean	max	min
fuel						
CNG	66050.396226	270000	1	313415.037736	545000	80000
Diesel	83319.920214	1500000	1000	802450.282624	7200000	45000
LPG	88856.685714	300000	30000	210885.714286	375000	54000
Petrol	51790.497727	2360457	1000	472665.203125	10000000	29999

Grouping by fuel and transmission

[122]:
```
df.groupby(by=['fuel', 'transmission'])['selling_price'].agg(
    ['mean', 'count', 'max', 'min']
)
```

Task 2. Joins

Create a second DataFrame with a common key column. Combine the two DataFrames using merge() or join(). Perform inner and left joins and compare the resulting DataFrames.

Since the dataset did not have a column with a unique identifier for each row, I used join to merge by index, having first created 2 DataFrames with different columns.

[123]:
```
df_additional = pd.read_csv("./archive/car_details_v3.csv", usecols=['name', 'torque', 'seats'])
```

[124]:
```
df_additional.head()
```

[124]:

	name	torque	seats
0	Maruti Swift Dzire VDI	190Nm@ 2000rpm	5.0
1	Skoda Rapid 1.5 TDI Ambition	250Nm@ 1500-2500rpm	5.0
2	Honda City 2017-2020 EXi	12.7@ 2,700(kgm@ rpm)	5.0
3	Hyundai i20 Sportz Diesel	22.4 kgm at 1750-2750rpm	5.0
4	Maruti Swift VXI BSIII	11.5@ 4,500(kgm@ rpm)	5.0

[125]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 7907 entries, 0 to 8127
Data columns (total 11 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   name           7907 non-null   str
 1   year           7907 non-null   int64
 2   selling_price  7907 non-null   int64
 3   km_driven      7907 non-null   int64
 4   fuel           7907 non-null   category
 5   seller_type    7907 non-null   category
```

[126]:
```
df_merged_left = df.join(df_additional, lsuffix='_caller', rsuffix='_other')
```

[127]:
```
df_merged_left.head()
```

[127]:

	name_caller	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	name_other	torque	seats
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Individual	Manual	First Owner	23.40	1248	74 bhp	Maruti Swift Dzire VDI	190Nm@ 2000rpm	5.0
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Individual	Manual	Second Owner	21.14	1498	103.52 bhp	Skoda Rapid 1.5 TDI Ambition	250Nm@ 1500-2500rpm	5.0
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	Individual	Manual	Third Owner	17.70	1497	78 bhp	Honda City 2017-2020 EXi	12.7@ 2,700(kgm@ rpm)	5.0
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	Individual	Manual	First Owner	23.00	1396	90 bhp	Hyundai i20 Sportz Diesel	22.4 kgm at 1750-2750rpm	5.0
4	Maruti Swift VXI BSIII	2007	130000	120000	Petrol	Individual	Manual	First Owner	16.10	1298	88.2 bhp	Maruti Swift VXI BSIII	11.5@ 4,500(kgm@ rpm)	5.0

[128]:
```
df_merged_left.shape
```

[128]:
```
(7907, 14)
```

[129]:
```
df_merged_inner = df.join(df_additional, lsuffix='_caller', rsuffix='_other', how="inner")
```

```
[130]: df_merged_right = df.join(df_additional, lsuffix='_caller', rsuffix='_other', how="right")
[131]: df_merged_left.equals(df_merged_right)
[131]: False
[132]: df_merged_left.shape
[132]: (7907, 14)
[133]: df_merged_right.shape
[133]: (8128, 14)
[134]: df_merged_right.iloc[200:206]
```

[134]:

	name_caller	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	name_other	torque	seats
200	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Toyota Etios GD	NaN	NaN
201	Honda City i VTEC S	2015.0	600000.0	50000.0	Petrol	Individual	Manual	First Owner	17.400	1497.0	117.3 bhp	Honda City i VTEC S	145Nm@ 4600rpm	5.0
202	Honda Brio S MT	2014.0	290000.0	60000.0	Petrol	Individual	Manual	Second Owner	19.400	1198.0	86.8 bhp	Honda Brio S MT	109Nm@ 4500rpm	5.0
203	Maruti Wagon R LXI CNG	2016.0	360000.0	50000.0	CNG	Individual	Manual	First Owner	19.152	998.0	58.16 bhp	Maruti Wagon R LXI CNG	77Nm@ 3500rpm	5.0

If there were null values, left and right joins would have number of rows present in corresponding left and right DataFrames. Inner join holds not null values (rows) from both DataFrames. As I didn't delete null containing rows from df_additional, performing right join saved those values, providing null to all columns from df.

Task 3. Reshaping

Create a pivot_table() using suitable categorical and numerical columns. Then reshape part of the dataset from wide to long format using melt(). Display the transformed DataFrames.

```
[136]: pivot_table = df.pivot_table(index='year', columns='fuel', values='selling_price', aggfunc="mean")
       pivot_table
```

[136]:

fuel	CNG	Diesel	LPG	Petrol
year				
1994	NaN	9.900000e+04	NaN	4.500000e+04
1995	NaN	NaN	NaN	5.500000e+04
1996	NaN	NaN	NaN	4.100000e+04
1997	NaN	NaN	NaN	8.611100e+04
1998	NaN	NaN	NaN	5.788889e+04
1999	NaN	1.516667e+05	NaN	5.654545e+04
2000	80000.000000	1.066667e+05	NaN	6.890950e+04
2001	NaN	NaN	NaN	4.650000e+04
2002	NaN	1.486663e+05	NaN	9.537500e+04
2003	NaN	1.478000e+05	NaN	8.445178e+04
2004	NaN	1.280000e+05	NaN	1.048745e+05
2005	NaN	2.273182e+05	NaN	1.097129e+05
2006	NaN	2.451500e+05	110000.000000	1.397556e+05

[137]:
```
pivot_table.reset_index().melt(id_vars=['year'], var_name='fuel' ,value_name='selling_price_avg').dropna().head()
```

[137]:

	year	fuel	selling_price_avg
6	2000	CNG	80000.000000
14	2008	CNG	120000.000000
16	2010	CNG	188333.333333
17	2011	CNG	198500.000000
18	2012	CNG	221400.000000

[138]:
```
df[df['fuel'] == 'CNG'][['year', 'fuel', 'selling_price']].groupby(by='year')[['year', 'fuel', 'selling_price']].agg(
    selling_price_avg = ('selling_price', 'mean')
).head()
```

[138]:

	selling_price_avg
year	
2000	80000.000000
2008	120000.000000
2010	188333.333333
2011	198500.000000
2012	221400.000000

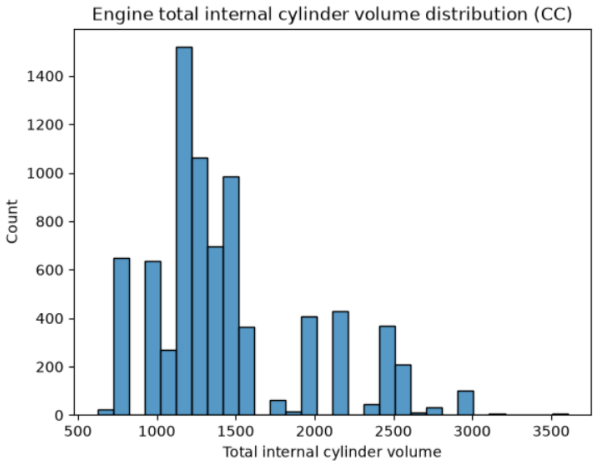
Task 4. Visualization

Create at least four plots using Matplotlib and Seaborn:

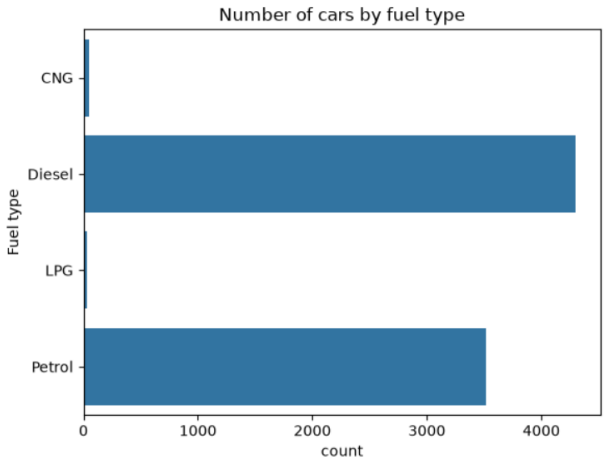
- distribution of a numerical variable;
- comparison of categories;
- relationship between two numerical variables;
- comparison of a numerical variable across categories.

Add titles and axis labels to all plots.

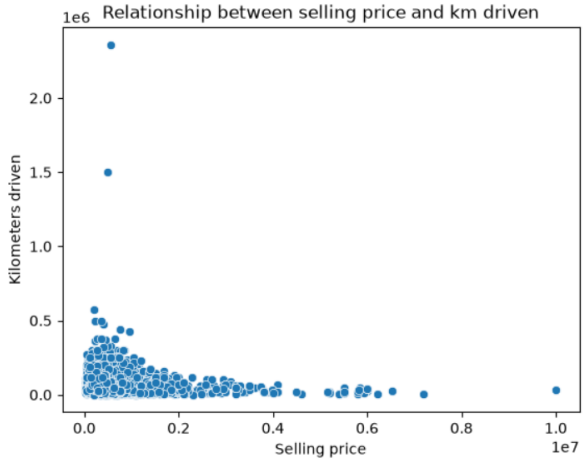
[139]:
```
sns.histplot(df['engine'], binwidth=100)
plt.title('Engine total internal cylinder volume distribution (CC)')
plt.xlabel('Total internal cylinder volume')
plt.show()
```



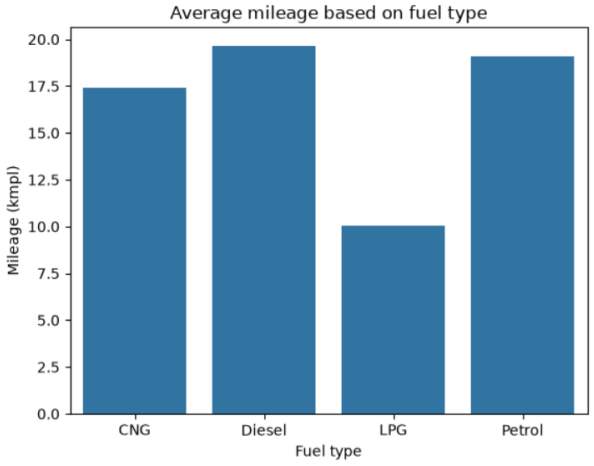
[140]:
```
sns.countplot(df['fuel']);
plt.title('Number of cars by fuel type')
plt.ylabel('Fuel type')
plt.show()
```



[141]:
```
sns.scatterplot(df, x='selling_price', y='km_driven')
plt.title('Relationship between selling price and km driven')
plt.xlabel('Selling price')
plt.ylabel('Kilometers driven')
plt.show()
```



[142]:
```
sns.barplot(df, x='fuel', y='mileage', estimator='mean', errorbar=None)
plt.title('Average mileage based on fuel type')
plt.xlabel('Fuel type')
plt.ylabel("Mileage (kmpl)")
plt.show()
```



Task 5. EDA

Perform an exploratory analysis using grouped statistics and visualizations. Compare categories, identify patterns in numerical variables, and examine relationships between selected features.Write five findings supported by the calculated results and plots.

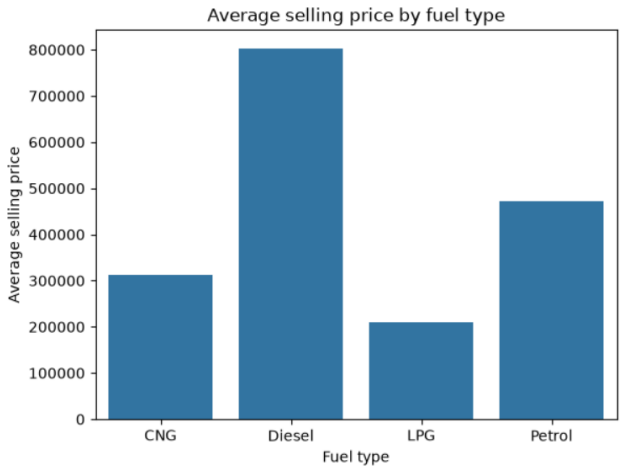
1. Average selling price by fuel

[143]:
```
df.groupby('fuel')['selling_price'].agg(['mean', 'median', 'count'])
```

[143]:

fuel	mean	median	count
CNG	313415.037736	330000.0	53
Diesel	802450.282624	550000.0	4299
LPG	210885.714286	200000.0	35
Petrol	472665.203125	350000.0	3520

[144]:
```
sns.barplot(df, x='fuel', y='selling_price', estimator='mean', errorbar=None
)
plt.title('Average selling price by fuel type')
plt.xlabel('Fuel type')
plt.ylabel('Average selling price')
plt.show()
```



Finding

The average selling price differs between fuel types. Among 4 fuel types, diesel has the highest average selling price and LPG the lowest.

2. Mileage accross fuel types

Visualization

Finding

Diesel has the best mileage in kilometers per liter of fuel.

3. Selling price vs kilometers driven

Visualization

[145]:
```
df[['selling_price', 'km_driven']].corr()
```

[145]:

	selling_price	km_driven
selling_price	1.000000	-0.222196
km_driven	-0.222196	1.000000

Finding

There is a weak negative correlation between kilometers driven and car selling price. This mean the more kilometers were traveled, the chepear selling price is.

4. Selling price accross years

[146]:
```
df.groupby(by='year').agg(
    selling_price_avg = ('selling_price', 'mean'),
    selling_price_median = ('selling_price', 'median'),
    selling_price_count = ('selling_price', 'count')
)
```

[146]:

year	selling_price_avg	selling_price_median	selling_price_count
1994	7.200000e+04	72000.0	2
1995	5.500000e+04	55000.0	1
1996	4.100000e+04	41000.0	2
1997	8.611100e+04	50000.0	9
1998	5.788889e+04	58000.0	9
1999	7.692857e+04	62500.0	14
2000	7.668212e+04	77500.0	16
2017	8.901930e+05	645000.0	1010
2018	9.581836e+05	715000.0	806
2019	1.776986e+06	779000.0	583
2020	8.852702e+05	590000.0	74

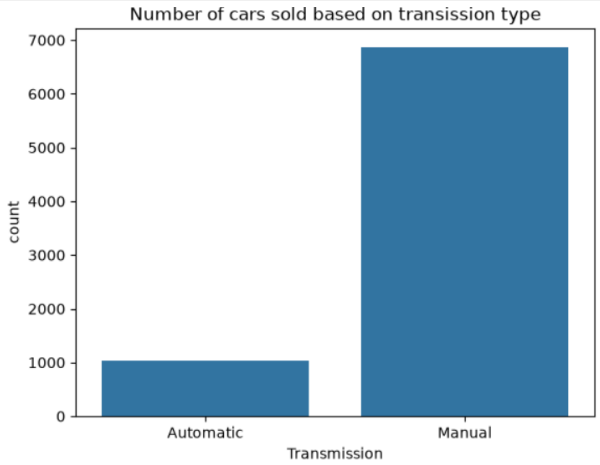
Finding

1. The largest number of cars was sold in 2017 — 1,010
2. The highest median price occured in 2019
3. The highest average price occured in 2019

5. Sales count depending on the transmission

[147]:
```
sns.countplot(df, x='transmission')

plt.title('Number of cars sold based on transission type')
plt.xlabel('Transmission')
plt.show()
```



Finding

Manual transission is far more popular than Automatic.